Structural Design

NAMA: Rahadyan Rakha Dzakwan Putra

KELAS: 1 D4 Teknik Komputer B

NRP: 3223600059

DOSEN: Reni Soelistijorini B.Eng, MT

MATA KULIAH: Rangkaian Logika

TANGGAL: 2 Mei 2024

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2023

A. TUJUAN

- Dapat membuat program dengan model disain structural.
- Dapat mengimplementasikan algoritma multiplexer pada disain structural.

B. ALAT DAN BAHAN

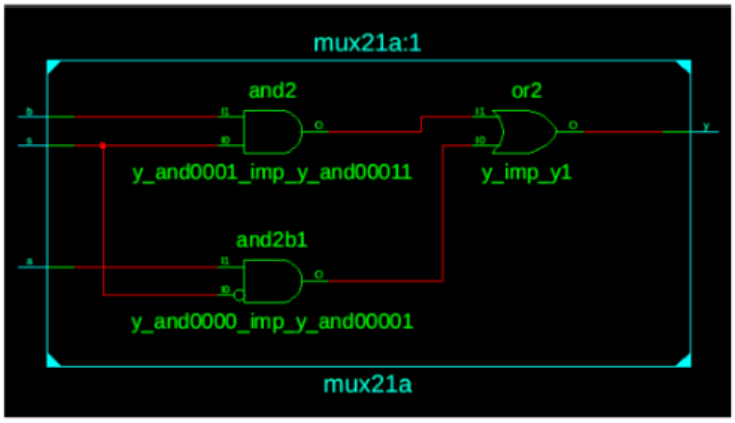
1. PC yang sudah terinstal ISE 13.1
2. Xilinx Spartan 3
3. Downloader JTAG USB
4. Power Supply 5 volt

C. PERCOBAAN

1. Multiplexer 2x1 tanpa structural design
   a. Program VHDL Module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux21a is
        port( a : in std_logic;
              b : in std_logic;
              s : in std_logic;
              y : out std_logic);
end mux21a;

architecture Behavioral of mux21a is
begin
        y <= (not s and a) or (s and b);
end Behavioral;
```

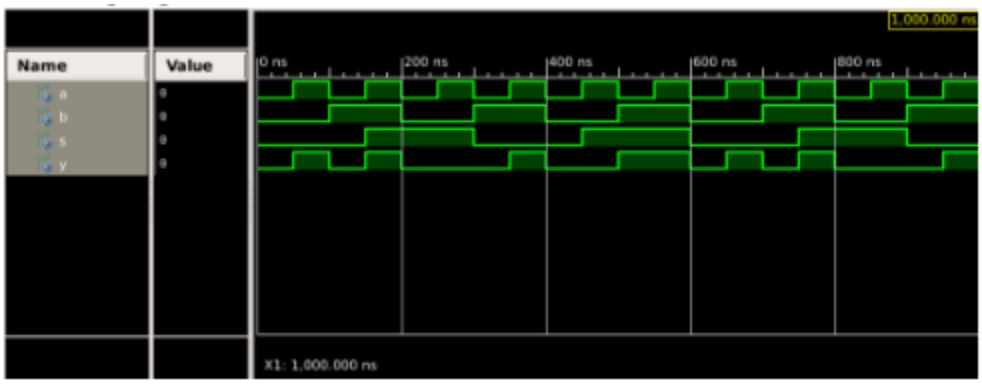
   b. RTL Schematic



c. Program VHDL Testbench

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mux21a_tb IS
END mux21a_tb;

ARCHITECTURE behavior OF mux21a_tb IS
	COMPONENT mux21a
	PORT(
		a : IN std_logic;
		b : IN std_logic;
		s : IN std_logic;
		y : OUT std_logic
		);
	END COMPONENT;

	signal a : std_logic := '0';
	signal b : std_logic := '0';
	signal s : std_logic := '0';
	signal y : std_logic;

BEGIN
	uut: mux21a PORT MAP (
		a => a,
		b => b,
		s => s,
		y => y
		);
	stim_proc: process
	begin
		wait for 50 ns;
			a <= not a;
	end process;
	stim_proc1: process
	begin
		wait for 100 ns;
			b <= not b;
	end process;
	stim_proc2: process
	begin
		wait for 150 ns;
			s <= not s;
	end process;
	END;
```

d. Timing Diagram



e. Program pin.ucf

```
NET a LOC=K13;
NET b LOC=K14;
NET s LOC=J13;
NET y LOC=P11;
```

2. Multiplexer 2x1 dengan structural design

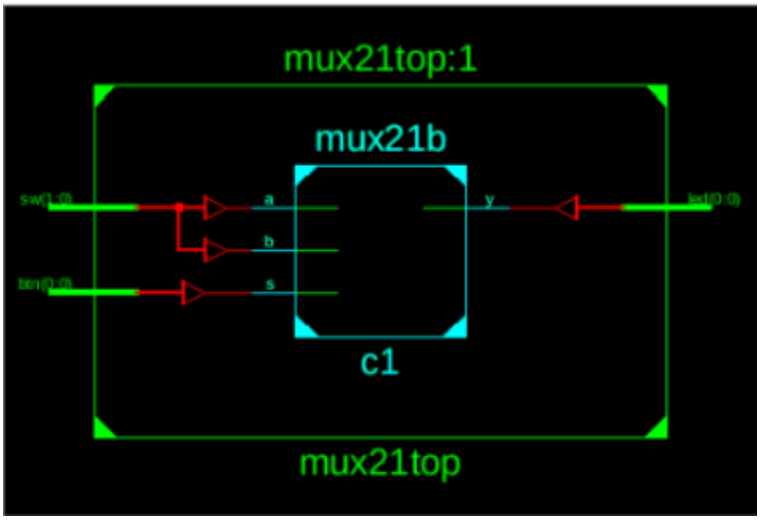
a. Program VHDL Module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux21b is
        port( a : in std_logic;
              b : in std_logic;
              s : in std_logic;
              y : out std_logic);
end mux21b;

architecture Behavioral of mux21b is
begin
        p1 : process (a, b, s)
          begin
                if s = '0' then
                  y <= a;
                else
                  y <= b;
                end if;
        end process;
end Behavioral;
```

b. Pogram VHDL Module Top Level

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux21top is
      port ( sw : in std_logic_vector (1 downto 0);
             btn : in std_logic_vector (0 downto 0);
              led : out std_logic_vector (0 downto 0));
end mux21top;

architecture Behavioral of mux21top is
     component mux21b is
          port ( a : in std_logic;
                 b : in std_logic;
                  s : in std_logic;
                  y : out std_logic);
      end component;

begin
            c1: mux21b port map (
                        a => sw(1),
                        b => sw(0),
                        s => btn(0),
                        y => led(0)
                        );
end Behavioral;
```

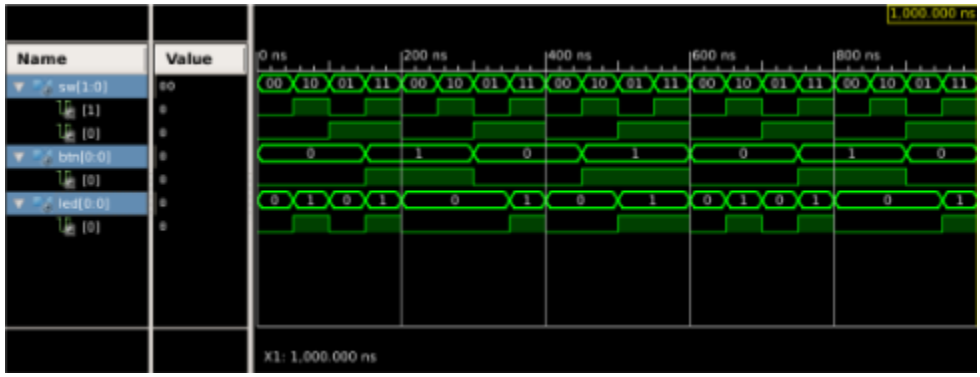
c. RTL Schematic



d. Program VHDL Testbench Top Level

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mux21top_tb IS
END mux21top_tb;
ARCHITECTURE behavior OF mux21top_tb IS
	COMPONENT mux21top
	PORT(
		sw : IN std_logic_vector(1 downto 0);
		btn : IN std_logic_vector(0 downto 0);
		led : OUT std_logic_vector(0 downto 0)
		);
	END COMPONENT;
	signal sw : std_logic_vector(1 downto 0) := (others => '0');
	signal btn : std_logic_vector(0 downto 0) := (others => '0');
	signal led : std_logic_vector(0 downto 0);
BEGIN
	uut: mux21top PORT MAP (
		sw => sw,
		btn => btn,
		led => led
	);
stim_proc: process
begin
	wait for 50 ns;
		sw(1) <= not sw(1);
end process;
stim_proc1: process
begin
	wait for 100 ns;
		sw(0) <= not sw(0);
end process;
stim_proc2: process
begin
	wait for 150 ns;
		btn <= not btn;
end process;
```

e. Timing Diagram



f. Program pin.ucf

```
NET sw(1) LOC=K13;
NET sw(0) LOC=K14;
NET btn(0) LOC=J13;
NET led(0) LOC=P11;
```

3. Full Adder 4 bit dengan menggunakan Full Adder 1 bit

a. Program VHDL Module

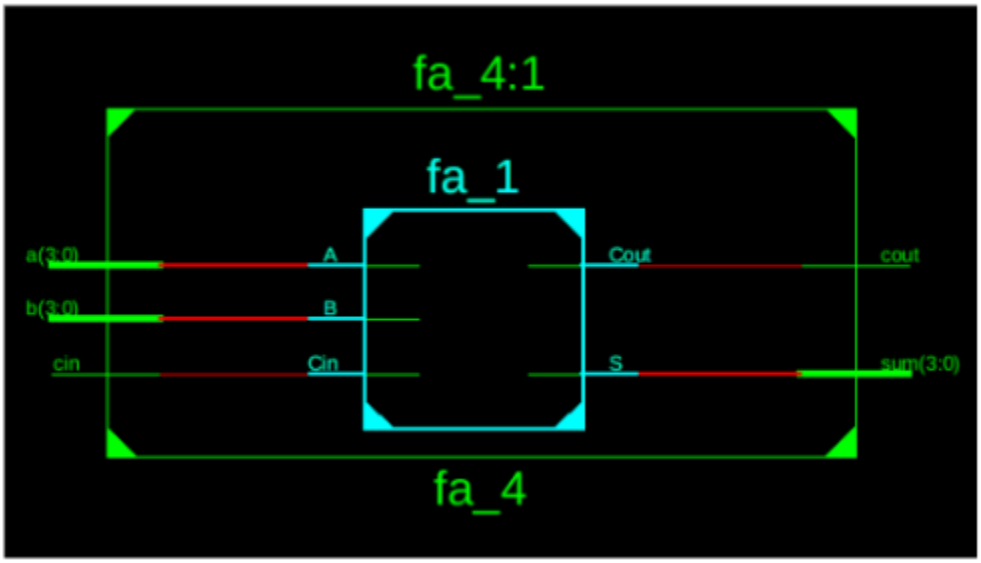
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fa_1 is
        port ( A: in std_logic;
               B: in std_logic;
               Cin: in std_logic;
               S : out std_logic;
               Cout : out std_logic);
end fa_1;
architecture Behavioral of fa_1 is
begin
        S <= A xor B xor Cin;
            Cout <= (A and B) or (A and Cin) or (B and Cin);
end Behavioral;
```

b. Program VHDL Module Top Level

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fa_4 is
        port ( a : in std_logic_vector (3 downto 0);
               b : in std_logic_vector (3 downto 0);
              cin : in std_logic;
              sum : out std_logic_vector (3 downto 0);
              cout : out std_logic);
end fa_4;
architecture Behavioral of fa_4 is
    component fa_1
        port ( A, B, Cin : in std_logic;
               S, Cout : out std_logic);
```

```
end component;
signal ci : std_logic_vector (4 downto 0);

begin
        ci(0) <= cin;
        u0 : fa_1 port map (A => a(3), B => b(3), Cin => ci(3), S =>
sum(3), Cout => ci(4));
        u1 : fa_1 port map (A => a(2), B => b(2), Cin => ci(2), S =>
sum(2), Cout => ci(3));
        u2 : fa_1 port map (A => a(1), B => b(1), Cin => ci(1), S =>
sum(1), Cout => ci(2));
        u3 : fa_1 port map (A => a(0), B => b(0), Cin => ci(0), S =>
sum(0), Cout => ci(1));
        cout <= ci(4);
end Behavioral;
```

c. RTL Schematic

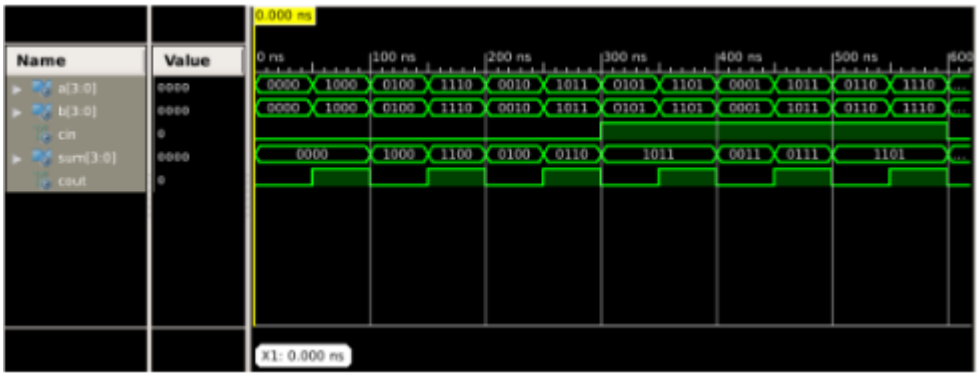


d. Program VHDL

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY fa_4_tb IS
END fa_4_tb;

ARCHITECTURE behavior OF fa_4_tb IS
        COMPONENT fa_4
            PORT(
            a : IN std_logic_vector(3 downto 0);
            b : IN std_logic_vector(3 downto 0);
            cin : IN std_logic;
            sum : OUT std_logic_vector(3 downto 0);
            cout : OUT std_logic
            );
        END COMPONENT;

    signal a : std_logic_vector(3 downto 0) := (others => '0');
    signal b : std_logic_vector(3 downto 0) := (others => '0');
    signal cin : std_logic := '0';
    signal sum : std_logic_vector(3 downto 0);
    signal cout : std_logic;

BEGIN
    uut: fa_4 PORT MAP (
            a => a,
            b => b,
            cin => cin,
            sum => sum,
            cout => cout
            );
    stim_proc: process
    begin
      wait for 50 ns;
            a(3) <= not a(3);
            b(3) <= not b(3);
    end process;

    stim_proc1: process
    begin
       wait for 100 ns;
            a(2) <= not a(2);
            b(2) <= not b(2);
    end process;
```

```
        stim_proc2: process
        begin
          wait for 150 ns;
                a(1) <= not a(1);
                b(1) <= not b(1);
        end process;

        stim_proc3: process
        begin
          wait for 250 ns;
                a(0) <= not a(0);
                b(0) <= not b(0);
        end process;

        stim_proc4: process
        begin
          wait for 300 ns;
                cin <= not cin;
        end process;
END;
```

e. Timing Diagram



4. Multiplexer 4x1 dan Demultiplexer 1x4

a. Program VHDL Module multiplexer 4x1

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux41 is
    port (a, b, c, d : in std_logic;
          s : in std_logic_vector (1 downto 0);
          v : out std_logic);
end mux41;

architecture Behavioral of mux41 is
begin
   p1 : process (a, b, c, d, s)
          begin
                if s(1) = '0' and s(0) = '0' then
                    v <= d;
                elsif s(1) = '0' and s(0) = '1' then
                    v <= c;
                elsif s(1) = '1' and s(0) = '0' then
                    v <= b;
                else
                    v <= a;
                end if;
          end process;
end Behavioral;
```

b. Program VHDL Module Demultiplexer 1x4

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity demux14 is
          port (
                    v : in std_logic;
                    s : in std_logic_vector (1 downto 0);
                    w, x, y, z : out std_logic
                    );
end demux14;
architecture Behavioral of demux14 is
begin
     process (v, s)
      begin
          w <= '0';
          x <= '0';
          y <= '0';
```

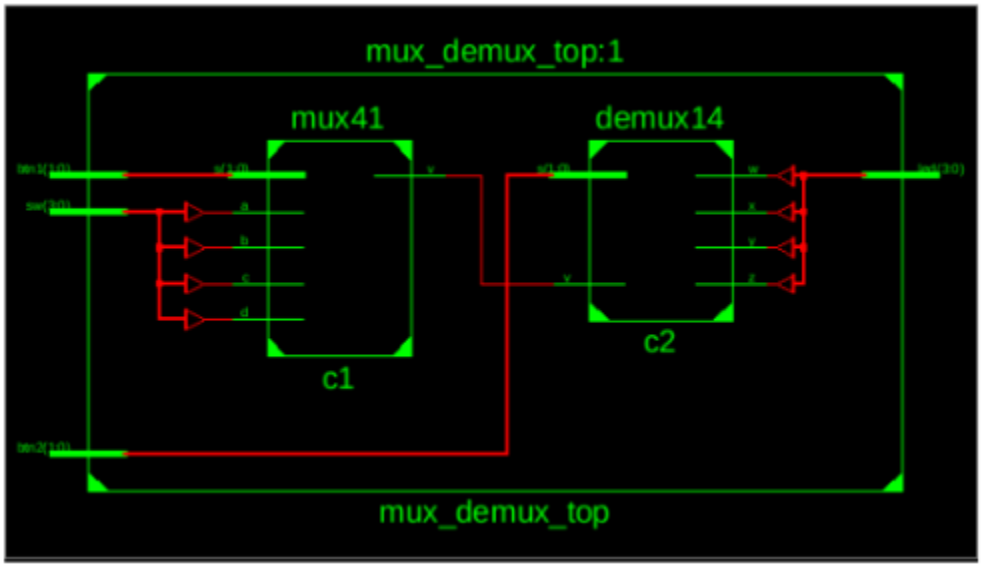
```
            z <= '0';
            if s = "00" then
                        z <= v;
            elsif s = "01" then
                        y <= v;
            elsif s = "10" then
                        x <= v;
            else
                        w <= v;
            end if;
        end process;
end Behavioral;
```

c. Program VHDL Module top level

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux_demux_top is
    port ( sw : in std_logic_vector (3 downto 0);
           btn1 : in std_logic_vector (1 downto 0);
           btn2 : in std_logic_vector (1 downto 0);
           led : out std_logic_vector (3 downto 0));
end mux_demux_top;

architecture Behavioral of mux_demux_top is
    component mux41 is
           port (a, b, c, d : in std_logic;
           s : in std_logic_vector (1 downto 0);
           v : out std_logic);
        end component;

        component demux14 is
           port (v : in std_logic;
           s : in std_logic_vector (1 downto 0);
           w, x, y, z : out std_logic);
        end component;

        signal connect : std_logic;
```

```
begin
   c1: mux41 port map (
            a => sw(3),
            b => sw(2),
            c => sw(1),
            d => sw(0),
            s(1) => btn1(1),
            s(0) => btn1(0),
            v => connect
);
    c2: demux14 port map (
            w => led(3),
            x => led(2),
            y => led(1),
            z => led(0),
            s(1) => btn2(1),
            s(0) => btn2(0),
            v => connect
);
end Behavioral;
```

d. RTL Schematic

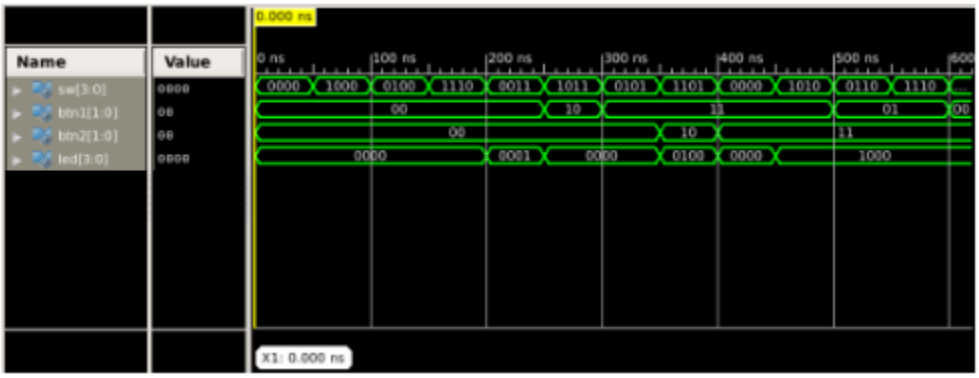


e. Program VHDL Testbench top level

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mux_demux_top_tb IS
END mux_demux_top_tb;

ARCHITECTURE behavior OF mux_demux_top_tb IS
    COMPONENT mux_demux_top
        PORT(
                sw : IN std_logic_vector(3 downto 0);
                btn1 : IN std_logic_vector(1 downto 0);
                btn2 : IN std_logic_vector(1 downto 0);
                led : OUT std_logic_vector(3 downto 0)
                );
    END COMPONENT;
        signal sw : std_logic_vector(3 downto 0) := (others => '0');
        signal btn1 : std_logic_vector(1 downto 0) := (others => '0');
        signal btn2 : std_logic_vector(1 downto 0) := (others => '0');
        signal led : std_logic_vector(3 downto 0);
BEGIN
    uut: mux_demux_top PORT MAP (
        sw => sw,
        btn1 => btn1,
        btn2 => btn2,
        led => led
        );

stim_proc: process
begin
        wait for 50 ns;
                sw(3) <= not sw(3);
end process;
stim_proc1: process
begin
        wait for 100 ns;
                sw(2) <= not sw(2);
end process;
stim_proc2: process
begin
        wait for 150 ns;
                sw(1) <= not sw(1);
end process;
```

```
stim_proc3: process
begin
        wait for 200 ns;
                sw(0) <= not sw(0);
end process;
stim_proc4: process
begin
        wait for 250 ns;
                btn1(1) <= not btn1(1);
end process;
stim_proc5: process
begin
        wait for 300 ns;
                btn1(0) <= not btn1(0);
end process;
stim_proc6: process
begin
        wait for 350 ns;
                btn2(1) <= not btn2(1);
end process;
stim_proc7: process
begin
        wait for 400 ns;
                btn2(0) <= not btn2(0);
end process;
END;
```

f. Timing Diagram



g. Program pin.ucf

```
NET sw(3) LOC=K13;
NET sw(2) LOC=K14;
NET sw(1) LOC=J13;
NET sw(0) LOC=J14;
NET btn1(1) LOC=H13;
NET btn1(0) LOC=H14;
NET btn2(1) LOC=M14;
NET btn2(0) LOC=F12;
NET led(3) LOC=P11;
NET led(2) LOC=P12;
NET led(1) LOC=N12;
NET led(0) LOC=P13;
```

5. HASIL

a. Multiplexer 2x1 tanpa structural design

b. Multiplexer 2x1 dengan structural design

c. Full Adder 4 bit dengan menggunakan Full Adder 1 bit

d. Multiplexer 4x1 dan Demultiplexer 1x4

6. ANALISA

Pada program Multiplexer 2x1 tanpa structural design, input s digunakan sebagai selektor, sedangkan input a dan b adalah dua sinyal yang mungkin dipilih. Keluaran y diatur menggunakan ekspresi boolean yang bergantung pada nilai input s. Jika s bernilai '0', maka keluaran y akan sama dengan input a, dan jika s bernilai '1', keluaran y akan sama dengan input b. Meskipun program ini sederhana, itu memberikan fungsionalitas dasar dari multiplexer 2-ke-1.

Pada program Multiplexer 2x1 dengan structural design, entitas mux21b memiliki tiga input, yaitu a, b, dan s, yang mewakili dua sinyal input dan satu sinyal selektor, serta output y yang merupakan hasil dari pemilihan antara input a atau b berdasarkan nilai s. Proses di dalam mux21b mengatur output y berdasarkan nilai s: jika s bernilai '0', maka output y akan sama dengan input a, dan jika s bernilai '1', output y akan sama dengan input b. Entitas mux21top menggunakan komponen mux21b untuk membuat multiplexer 2-ke-1 yang dikendalikan oleh nilai dari switch (sw) dan tombol (btn). Nilai sw(1) dan sw(0) digunakan sebagai input a dan b pada mux21b, sedangkan btn(0) digunakan sebagai input s. Hasil dari multiplexer, yaitu output dari mux21b, ditampilkan pada led(0). Meskipun program ini sederhana, itu menunjukkan bagaimana sebuah komponen multiplexer dapat digunakan dan diintegrasikan dalam desain yang lebih besar.

Pada program Full Adder 4 bit dengan menggunakan Full Adder 1 bit, entitas fa_1 mendefinisikan satu full adder 1-bit dengan tiga input (A, B, Cin) dan dua output (S, Cout), di mana S adalah hasil penjumlahan satu bit dan Cout adalah carry out. Pada entitas fa_4, komponen fa_1 digunakan empat kali untuk mengimplementasikan penjumlahan 4-bit. Setiap full adder menerima input dari bit-bit yang sesuai dari vektor input a dan b, serta carry in (ci) dari iterasi sebelumnya. Carry in untuk iterasi pertama diinisialisasi dengan carry in global (cin). Setelah itu, carry out dari iterasi terakhir digunakan sebagai carry out global (cout). Hasil penjumlahan satu bit dari setiap full adder digunakan untuk membentuk vektor output sum, sementara carry out global (cout) dihasilkan dari carry out terakhir. Meskipun program ini membagi penjumlahan 4-bit menjadi penjumlahan 1-bit yang lebih kecil untuk mempermudah implementasi, pendekatan ini memungkinkan penambahan paralel dari bit-bit yang sesuai, meningkatkan kecepatan operasi.

Pada program Multiplexer 4x1 dan Demultiplexer 1x4, keluaran yang mungkin terdiri dari w, x, y, dan z, masing-masing menghubungkan ke salah satu output dari demultiplexer. Dalam arsitekturnya, proses tersebut digunakan untuk mengatur nilai keluaran w, x, y, dan z berdasarkan nilai s. Jika s adalah "00", maka v akan diarahkan ke z. Jika s adalah "01", v akan diarahkan ke y. Jika s adalah "10", v akan diarahkan ke x. Jika s bukan "00", "01", atau "10", maka v akan diarahkan ke w. Meskipun program ini sederhana, itu memberikan fungsionalitas dasar dari demultiplexer 1-ke-4.